

Resumen

El presente informe describe el desarrollo de RelevApp, una aplicación web progresiva (PWA) orientada a digitalizar el proceso de relevamiento de campo que realizan técnicos de servicios mediante la captura de imágenes, el reconocimiento óptico de caracteres (OCR) y el registro de la ubicación geográfica del relevamiento. Se detallan los objetivos del proyecto, los requisitos relevados, la arquitectura adoptada, las tecnologías utilizadas y las pruebas realizadas sobre el sistema. El desarrollo se llevó a cabo utilizando tecnologías web estándar (HTML, CSS y JavaScript), sin requerir un servidor de aplicación propio, lo que permitió reducir los costos de infraestructura y simplificar el despliegue.

Palabras clave: aplicación web progresiva, reconocimiento óptico de caracteres, geolocalización, relevamiento de campo.

Introducción

Las empresas que prestan servicios técnicos en distintos puntos geográficos suelen depender de procesos manuales para registrar la información obtenida durante un relevamiento de campo. Esta modalidad de trabajo presenta limitaciones conocidas: errores de transcripción, demoras en la carga de datos y dificultad para verificar que la tarea fue efectivamente realizada en el lugar correspondiente.

Frente a esta problemática, se desarrolló RelevApp, una aplicación web progresiva que permite a un técnico capturar una imagen con la cámara del dispositivo, extraer automáticamente el texto presente en ella mediante reconocimiento óptico de caracteres y registrar la ubicación geográfica exacta del relevamiento sobre un mapa interactivo. Al tratarse de una aplicación web progresiva, RelevApp puede instalarse en el dispositivo del usuario sin pasar por una tienda de

aplicaciones y continúa funcionando aun sin conexión a internet una vez completada la primera carga.

En las secciones siguientes se presentan los objetivos del proyecto, el relevamiento de requisitos funcionales y no funcionales, la arquitectura de la solución, las tecnologías empleadas, las pruebas realizadas y las conclusiones obtenidas a partir del desarrollo.

Objetivos

Objetivo general

Desarrollar una aplicación web progresiva que permita capturar texto desde imágenes mediante reconocimiento óptico de caracteres y registrar la ubicación geográfica del relevamiento, garantizando su funcionamiento tanto en línea como sin conexión a internet.

Objetivos específicos

- Permitir la captura de imágenes mediante la cámara del dispositivo o la selección desde la galería.
- Extraer el texto presente en la imagen capturada mediante reconocimiento óptico de caracteres.
- Informar al usuario el nivel de confianza del texto reconocido.
- Permitir copiar el texto extraído al portapapeles del dispositivo.
- Obtener la ubicación geográfica actual del usuario mediante la API de geolocalización del navegador.
- Representar la ubicación obtenida sobre un mapa interactivo.
- Implementar la aplicación como una PWA instalable en dispositivos móviles y de escritorio.

- Garantizar la continuidad de funcionamiento de la aplicación sin conexión a internet.

Relevamiento de requisitos

Requisitos funcionales

A continuación se enumeran los requisitos funcionales identificados para el sistema:

- El sistema debe permitir capturar una imagen utilizando la cámara del dispositivo.
- El sistema debe permitir seleccionar una imagen existente desde la galería del dispositivo.
- El sistema debe preprocesar la imagen capturada (conversión a escala de grises, ajuste de contraste y binarización) antes de aplicar el reconocimiento óptico de caracteres.
- El sistema debe extraer el texto contenido en la imagen mediante un motor de reconocimiento óptico de caracteres.
- El sistema debe mostrar el texto reconocido junto con el porcentaje de confianza del reconocimiento.
- El sistema debe permitir copiar el texto reconocido al portapapeles.
- El sistema debe obtener la latitud, la longitud y la precisión de la ubicación actual del usuario.
- El sistema debe representar la ubicación obtenida sobre un mapa interactivo, incluyendo un marcador visual.
- El sistema debe poder instalarse como aplicación en los dispositivos compatibles.

- El sistema debe continuar funcionando, al menos parcialmente, sin conexión a internet.

Requisitos no funcionales

Se identificaron además los siguientes requisitos no funcionales:

- Usabilidad: la interfaz debe poder operarse con una sola mano durante el trabajo de campo.
- Rendimiento: el procesamiento de OCR debe ejecutarse en el dispositivo del usuario, sin enviar las imágenes a un servidor externo.
- Portabilidad: la aplicación debe funcionar en cualquier navegador moderno compatible con el estándar de aplicaciones web progresivas.
- Disponibilidad: la aplicación debe permanecer utilizable sin conexión a internet luego de la primera carga.
- Privacidad: las imágenes capturadas y los datos de ubicación no deben transmitirse a servidores de terceros.
- Mantenibilidad: el código debe organizarse en módulos independientes según su responsabilidad.

Arquitectura de la solución

RelevApp fue desarrollada siguiendo una arquitectura del lado del cliente, sin backend propio. La totalidad de la lógica de negocio (preprocesamiento de imágenes, reconocimiento óptico de caracteres, obtención de coordenadas y renderizado del mapa) se ejecuta en el navegador del dispositivo del usuario. La aplicación consume únicamente recursos externos estáticos: las

librerías de terceros (a través de una red de distribución de contenidos) y el servicio de mapas de OpenStreetMap.

El proyecto se organiza en cuatro capas funcionales. La primera, la capa de presentación, está compuesta por el archivo `index.html` y la hoja de estilos `style.css`, responsables de la estructura visual y de la navegación entre secciones de la aplicación. La segunda, la capa de control, corresponde al archivo `app.js`, que actúa como controlador principal y coordina el resto de los módulos en respuesta a las acciones del usuario. La tercera, la capa funcional, incluye los módulos `ocr.js`, `location.js` y `mapa.js`, cada uno con una responsabilidad única e independiente entre sí. Por último, la capa de persistencia offline está conformada por el archivo `service-worker.js`, encargado de almacenar en caché los recursos estáticos de la aplicación y de definir las estrategias de respuesta ante peticiones de red.

En cuanto al despliegue, el código fuente de la aplicación se aloja en un servidor de hosting estático, por ejemplo GitHub Pages. Cuando el usuario accede por primera vez, el navegador descarga los archivos necesarios mediante el protocolo HTTPS. A partir de ese momento, el Service Worker intercepta las peticiones de red y sirve los recursos desde la memoria caché del dispositivo, lo que permite que la aplicación continúe funcionando sin conexión a internet. La obtención de la ubicación geográfica se realiza directamente a través del sensor GPS del dispositivo mediante la API de Geolocalización del navegador, sin que estos datos pasen por ningún servidor intermedio.

Estructura del proyecto

El código fuente del proyecto se organiza de la siguiente manera: en la raíz del proyecto se encuentran los archivos `index.html`, `manifest.json` y `service-worker.js`. Dentro de la carpeta `css` se encuentra el archivo `style.css`. Dentro de la carpeta `js` se encuentran los archivos `app.js`, `ocr.js`,

location.js y mapa.js. Por último, dentro de la carpeta img se encuentra el ícono de la aplicación, y en la raíz del proyecto se incluye además un archivo README.md con la documentación general.

Casos de uso

El actor principal del sistema es el técnico de campo, quien interactúa con RelevApp para realizar las siguientes acciones: capturar una imagen, extraer el texto contenido en ella, copiar dicho texto, obtener su ubicación actual y visualizar esa ubicación sobre un mapa. A continuación se detalla la especificación de los casos de uso principales.

Caso de uso: Extraer texto mediante OCR

El actor principal de este caso de uso es el técnico de campo. Como precondition, el usuario debe haber capturado o seleccionado previamente una imagen. El flujo principal consiste en los siguientes pasos: el usuario captura o selecciona una imagen; el sistema preprocesa la imagen aplicando escala de grises, ajuste de contraste y binarización; el sistema ejecuta el motor de reconocimiento óptico de caracteres sobre la imagen procesada; finalmente, el sistema muestra el texto reconocido junto con el porcentaje de confianza obtenido. Como flujo alternativo, si no se detecta texto en la imagen, el sistema informa al usuario que no se encontró texto. Como postcondición, el texto reconocido queda disponible para que el usuario pueda copiarlo.

Caso de uso: Obtener ubicación

El actor principal de este caso de uso es el técnico de campo. Como precondition, el navegador debe contar con los permisos de geolocalización otorgados por el usuario. El flujo principal consiste en los siguientes pasos: el usuario presiona el botón de obtener ubicación; el sistema solicita la posición actual mediante la API de Geolocalización; el sistema muestra la

latitud, la longitud, la precisión y la fecha y hora del registro; finalmente, el sistema centra el mapa y coloca un marcador en la posición obtenida. Como flujo alternativo, si el usuario deniega el permiso o no hay señal disponible, el sistema muestra un mensaje de error explicativo. Como postcondición, la posición obtenida queda representada en el mapa mediante un marcador.

Tecnologías utilizadas

Para el desarrollo de RelevApp se utilizaron exclusivamente tecnologías web estándar del lado del cliente. La estructura de la aplicación fue construida con HTML5, mientras que el diseño visual y la disposición de los elementos se resolvieron mediante CSS3. La lógica de la aplicación se implementó en JavaScript, organizada en módulos independientes según su responsabilidad.

Para el reconocimiento óptico de caracteres se utilizó la librería Tesseract.js, que permite ejecutar el motor de OCR directamente en el navegador, sin necesidad de enviar las imágenes a un servidor externo. Para la obtención de la ubicación del usuario se utilizó la API de Geolocalización, nativa de los navegadores modernos. Para la representación del mapa interactivo se utilizó la librería Leaflet.js, en conjunto con los servicios de OpenStreetMap como proveedor de los datos cartográficos.

Finalmente, para habilitar el funcionamiento de la aplicación como una PWA se implementó un Service Worker, encargado de administrar el almacenamiento en caché de los recursos, y un archivo manifest.json, que define los metadatos necesarios para que el navegador permita instalar la aplicación en el dispositivo del usuario.

Pruebas realizadas

Se realizaron pruebas funcionales sobre las principales características del sistema. Se verificó la captura de imágenes tanto mediante la cámara del dispositivo como a través de la

selección de archivos desde la galería, obteniendo en ambos casos el resultado esperado. Se probó el reconocimiento óptico de caracteres sobre imágenes con buen contraste, obteniendo un porcentaje de confianza superior al ochenta por ciento, y sobre capturas de pantalla con texto de menor tamaño, donde el preprocesamiento de la imagen permitió mejorar de manera notoria la calidad del reconocimiento respecto de no aplicar dicho preprocesamiento.

Asimismo, se verificó el correcto funcionamiento de la copia de texto al portapapeles, la obtención de la ubicación con los permisos otorgados y con los permisos denegados, la representación del marcador sobre el mapa, la instalación de la aplicación como PWA en los navegadores Chrome y Edge, y el funcionamiento de la interfaz sin conexión a internet luego de la primera carga, gracias al almacenamiento en caché realizado por el Service Worker. En todos los casos mencionados, el resultado obtenido fue conforme a lo esperado.

Conclusiones

El desarrollo de RelevApp permitió aplicar de manera integrada los conceptos centrales de las aplicaciones web progresivas, entre ellos el archivo de manifiesto para la instalación, el Service Worker para el funcionamiento sin conexión, y el consumo de las API nativas del navegador para el acceso a la cámara y a la geolocalización, sin requerir un servidor de aplicación propio.

La incorporación de un preprocesamiento de imagen previo al reconocimiento óptico de caracteres resultó relevante para mejorar la precisión del sistema, particularmente en condiciones de iluminación desigual o de bajo contraste. Esto confirma la importancia de tratar adecuadamente los datos de entrada antes de aplicar algoritmos de reconocimiento automático.

En términos generales, el proyecto demuestra que es posible resolver una necesidad concreta de digitalización de procesos de campo utilizando exclusivamente tecnologías web estándar, sin incurrir en costos de infraestructura de servidor ni depender de la distribución a través

de tiendas de aplicaciones. Como posibles líneas de trabajo futuro se identifican la incorporación de almacenamiento local persistente para sincronizar los relevamientos una vez recuperada la conexión a internet, la implementación de un sistema de autenticación de usuarios y la exportación de los datos relevados en formatos estructurados.

Referencias

Google Developers. (2024). Progressive Web Apps. <https://web.dev/explore/progressive-web-apps>

Leaflet. (2024). Leaflet: an open-source JavaScript library for interactive maps.
<https://leafletjs.com/>

Mozilla Developer Network. (2024a). Canvas API.
https://developer.mozilla.org/en-US/docs/Web/API/Canvas_API

Mozilla Developer Network. (2024b). Geolocation API.
https://developer.mozilla.org/en-US/docs/Web/API/Geolocation_API

Mozilla Developer Network. (2024c). Service Worker API. https://developer.mozilla.org/en-US/docs/Web/API/Service_Worker_API

Mozilla Developer Network. (2024d). Web App Manifest.
<https://developer.mozilla.org/en-US/docs/Web/Manifest>

OpenStreetMap Foundation. (2024). OpenStreetMap. <https://www.openstreetmap.org/>

Tesseract.js. (2024). Tesseract.js: Pure Javascript OCR for 100 languages.
<https://tesseract.projectnaptha.com/>